SOC L1 LAB · INC-006

Detection, Mitigation, and Triage Guide — INC-006

How to close a correct alert without hand-waving

INCIDENT	INC-006 — Authorized change triage
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	None (authorized change)

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. What the rules say, and what they do not
2. Triage model
3. Authorization
4. Integrity
5. Behaviour
6. Follow-on
7. Decision tree
8. False positive vs benign true positive
9. Closure matrix
10. Evidence standard for a "no incident" ticket
11. Tuning without suppressing
12. Common mistakes
13. Metrics
14. Coverage note

Audience: an L1 analyst who has a persistence alert and a change record, and an L2 who owns the rule.

This guide uses INC-006 as the worked example. The rules are 100129 and 100130 from INC-004.

---

1. What the rules say, and what they do not

Rule	Says	Does not say
100129	A file under `/etc/cron.d` appeared or changed	Whether it was approved
100130	A unit or enable link under `/etc/systemd/system` changed	Whether it was approved

Both are change monitors. A legitimate deployment and a malicious persistence file look identical at the file layer. The decision needs context.

---

2. Triage model

Four questions, in order:

#	Question	Evidence
1	**Authorization** — was there an approved change?	Change record, approver, window
2	**Integrity** — does the deployed file match the record?	SHA-256 comparison
3	**Behaviour** — does the content do what the change says?	`cat`, `systemctl cat`, `list-timers`
4	**Follow-on** — is there any sign of compromise?	Auth logs, FIM, process, egress

Close only when all four are satisfied. If any one fails, escalate.

---

3. Authorization

A change record must contain:

- a unique ID;
- requester and approver;
- a window that includes the deployment time;
- the exact file paths;
- the purpose and risk;
- a rollback.

Without a record, the alert is an unapproved persistence change, regardless of how benign the content looks.

3.1 What "approved" does not mean

- A verbal approval is not evidence.
- A ticket that lists a different file is not approval for this file.
- An old change window does not cover a new deployment.

---

4. Integrity

Compare every deployed file's SHA-256 to the value in the change record.

- **Match:** the file is the approved artifact.
- **Mismatch:** the deployed file is not what was approved. Escalate. Do not close on "the content looks fine".

In INC-006 all three hashes matched:

```
58acf8...  /etc/cron.d/log-maintenance
6a0555...  /etc/systemd/system/logrotate-check.service
83d683...  /etc/systemd/system/logrotate-check.timer
```

---

5. Behaviour

Read the content. The change record says "log cleanup and a health check". The files say:

- cron runs `find /var/log -name '*.gz' -mtime +7 -delete`;
- the service runs `/usr/bin/true`;
- the timer is daily.

No network call, no new account, no credential access. The content matches the purpose.

A change record that says "log cleanup" but a unit that runs `curl` is a mismatch, even if the hashes match the record. Verify both.

---

6. Follow-on

Check the window after the change:

- failed logins, invalid users, authentication failures;
- new accounts or sudoers changes;
- unexpected processes or outbound connections;
- additional files under the same directories.

In INC-006 there were none. That is what moves the case from "approved file" to "no incident".

---

7. Decision tree

```
Persistence alert
│
├─ Approved change record exists
│   ├─ Hashes match
│   │   ├─ Content matches the purpose
│   │   │   ├─ No follow-on anomalies → close as authorized change
│   │   │   └─ Follow-on anomalies    → escalate
│   │   └─ Content differs from purpose → escalate
│   └─ Hash mismatch → escalate
│
└─ No approved record
    ├─ Known package transaction → document and close
    ├─ Unknown but benign-looking → escalate (ask the owner)
    └─ Suspicious content/actor → incident
```

---

8. False positive vs benign true positive

This distinction is the core of the scenario.

Term	The alert was…	The activity was…	Action
False positive	wrong	irrelevant	fix the rule
Benign true positive	right	allowed	close with evidence
True positive	right	malicious	incident response

INC-006 is a **benign true positive / authorized change**. The files really did change. Closing it as a false positive would imply the rule malfunctioned and create pressure to suppress 100129/100130.

---

9. Closure matrix

Authorization	Integrity	Behaviour	Follow-on	Outcome
Approved	Match	Matches	None	Close — authorized
Approved	Match	Matches	Present	Escalate
Approved	Mismatch	—	—	Escalate
Approved	Match	Differs	—	Escalate
Missing	—	—	—	Escalate
Package transaction	Match	Known	None	Close with package ID

---

10. Evidence standard for a "no incident" ticket

A closure without evidence is not a closure. Include:

- change ID, approver, window;
- alert paths and timestamps;
- hash comparison result;
- content summary;
- timer/unit state;
- follow-on checks (auth, process, egress);
- the explicit decision and the rollback path.

INC-006 uses `docs/evidence/INC-006-change-record.md`, `INC-006-triage.txt`, and `INC-006-alerts.json`.

---

11. Tuning without suppressing

The wrong response to this scenario is "add these files to an ignore list". The right responses are:

- enrich the alert with the change ID when one exists;
- route approved changes to a queue that expects them;
- allow-list approved unit names while keeping the alert for new names;
- keep the hash check in the change record;
- review after the first scheduled run.

A suppression would hide the next unauthorized cron file. An allow-list with a change ID keeps the control visible.

---

12. Common mistakes

Mistake	Consequence
Calling a correct alert a false positive	Rule gets suppressed; future change missed
Closing on "the content looks fine"	No audit trail; a swapped file passes
Accepting a verbal approval	No evidence
Ignoring the deployment window	An old ticket covers a new change
Not checking follow-on activity	Misses a compromise hidden behind a real change
Removing the change without recording rollback	The next analyst cannot reconstruct state

---

13. Metrics

Metric	INC-006
Alerts	4 (1× 100129, 3× 100130)
Change record match	3/3 hashes
Follow-on anomalies	0
Time to close	~2 minutes (exercise)
Containment	none
Rule changes	none

The value is not speed alone; it is a closure that another analyst can audit.

---

14. Coverage note

The same method applies to:

- 100120 (`authorized_keys` change with a key-rotation ticket);
- 100127/100128 (a legitimate PHP deployment);
- package-installed cron files;
- any FIM rule that reports a change rather than an exploit.

The rules are different; the questions are the same.